AP Emlaki

Docker Runbook and Client Brief

Release: 1812-compatible Node.js port | Document language: English

This document explains how to run AP Emlaki with Docker and provides a customer-facing overview of the product, its workflows, security model, data storage and operational requirements.

Application URL http://localhost:3000

Docker container ap-emlaki

Docker image ap-emlaki:1812

Technology Node.js, Express, Prisma, SQLite and EJS

1. Quick Start with Docker

Install Docker Desktop and make sure its engine is running. Open PowerShell in the project directory and run:

```
cd "C:\Users\Davoodsina\Desktop\amlaki-lastUpdate"
docker compose up -d --build
```

The first run builds the image and creates persistent Docker volumes. Open http://localhost:3000 after the container becomes healthy. Future daily starts normally require only docker compose up -d.

2. Daily Administration Commands

```
docker compose up -d
docker compose ps
docker compose logs -f app
docker compose restart app
docker compose down
docker compose up -d --build
```

- Use docker compose up -d for a normal start.
- Use --build after changing application code, dependencies or Docker files.
- docker compose down removes the container and network but preserves named data volumes.
- The service uses restart: unless-stopped and starts automatically with Docker Desktop.

3. Configuration

Docker Compose reads optional overrides from .env. Start from .env.example and set a long random SESSION_SECRET before public deployment.

```
APP_PORT=3000
SESSION_SECRET=generate-a-long-random-secret
SESSION_COOKIE_SECURE=false
TRUST_PROXY=0
GEOCODING_PROVIDER=nominatim
GEOCODING_API_KEY=
```

- Set SESSION_COOKIE_SECURE=true when the site is delivered through HTTPS.
- Set TRUST_PROXY=1 only when a trusted reverse proxy terminates HTTPS.
- Change APP_PORT if port 3000 is already occupied, for example APP_PORT=8080.
- Never send or commit real passwords, API keys or session secrets.

4. Persistent Data

The deployment stores business data, sessions and uploaded files outside the container. Rebuilding or restarting the application therefore does not remove customer data.

Application data volume amlaki-lastupdate_app-data

Upload volume amlaki-lastupdate_app-uploads

Database inside container /app/data/dev.db

Session directory /app/data/sessions

The startup script never applies a potentially destructive schema conversion automatically. Existing databases are opened without modification. Review Prisma warnings and back up the data before applying schema changes manually:

```
npm run docker:schema
```

5. Backup and Recovery

Back up both named volumes on a regular schedule. The following PowerShell commands create compressed archives in the current directory:

```
docker run --rm -v amlaki-lastupdate_app-data:/data -v "${PWD}:/backup" alpine tar czf /
backup/ap-emlaki-data-backup.tar.gz -C /data .
docker run --rm -v amlaki-lastupdate_app-uploads:/data -v "${PWD}:/backup" alpine tar
czf /backup/ap-emlaki-uploads-backup.tar.gz -C /data .
```

For recovery, stop the application, restore each archive to the matching volume and start the service again. Keep dated backups outside the Docker host and test restoration periodically.

6. Health Check and Troubleshooting

Docker monitors the application through /healthz. A healthy response confirms that both the web application and SQLite database are available.

```
Invoke-WebRequest http://localhost:3000/healthz
```

Expected response: {"status":"ok","database":"ready"}. Useful diagnostic commands:

```
docker compose ps
docker compose logs --tail=200 app
docker volume ls
```

- If the site does not open, verify that the container is healthy and the configured port is free.
- If the container restarts, inspect application logs before recreating volumes.
- If login fails, verify that an active user exists and that the browser uses the current URL.

- Do not run a forced schema update without a current backup.

Customer Brief

7. Product Overview

AP Emlaki is a browser-based property-management platform for property managers, landlords and administrative teams. It consolidates operational, financial and communication workflows in one secured application. The current implementation is a source-compatible Node.js port of the INtex Hausverwaltung 1812 application.

- Properties and units: buildings, units, rooms, areas, occupancy, ownership, rent and operating information.
- Contacts: tenants, owners, suppliers, craftsmen, administrators and classifications.
- Contracts and administration: agreements, responsibilities, documents, notes, tasks and appointments.
- Accounting: bookings, account structures, recurring bookings, bank imports, cost allocation and SEPA XML.
- Documents and media: controlled upload, download, preview, thumbnails and full-text display.
- Reports and exports: CSV, Excel, Word, XML, PDF, DATEV and mail-merge formats.
- Dashboards and charts: seven dashboards and eighteen analytical chart definitions.

8. How the System Works

The browser sends requests to the Express application. Express validates the user session, checks CSRF tokens for changes, applies permission masks and restricts Team-scoped data. Metadata-driven engines render the majority of list, form, search, import, export, print, report, chart and dashboard pages. Prisma executes database operations against SQLite.

```
Browser
  -> Express routes and EJS views
  -> Authentication, CSRF, permissions and Team scope
  -> Metadata-driven and specialized business engines
  -> Prisma ORM
  -> SQLite + persistent upload/session volumes
```

Source metadata defines labels, fields, lookups, relationships, dashboards, charts, exports, reports and button actions. Generic engines provide consistent behavior, while specialized modules implement accounting, bank imports, SEPA, webhooks, geocoding and other business operations.

9. Typical User Workflow

1. The user signs in. The system loads the user group, access rights and Team. 2. The user opens a module such as Properties, Contacts or Units. 3. Lists and searches show only authorized records. 4. Forms create or update records and enforce field rules. 5. Related documents, notes, tasks, appointments and child records can be attached. 6. Authorized users run reports, exports, accounting actions and imports. 7. Audit logging records important changes.

10. Security Model

- Passwords are stored as bcrypt hashes for new and upgraded credentials.
- Sessions use HTTP-only and same-site cookies and persist in a file-backed Docker volume.
- All state-changing requests require a CSRF token.
- Access masks control read, add, edit, delete, export, print and import operations.

- Team filtering applies to lists, direct records, lookups, imports, files and accounting actions.
- Uploads are not exposed through an unrestricted public directory.
- Rich text is sanitized against an explicit allowlist.
- Download names, chart labels, markup and export values are escaped or validated.

11. Import, Export and Automation

The import wizard supports CSV, XLSX, vCard and iCalendar input with preview, field mapping, validation, duplicate policy and result summaries. Specialized bank imports reproduce the original booking side effects. Export engines provide office formats, PDF, DATEV and mail-merge datasets. Monthly recurring bookings run through a scheduled job, while webhooks and SMTP integrations are configured per Team.

12. Customer Responsibilities

- Keep Docker Desktop and the host operating system updated.
- Maintain off-host backups of application data and uploads.
- Use HTTPS and a trusted reverse proxy for internet-facing deployments.
- Protect administrator credentials, session secrets, SMTP passwords and API keys.
- Review user permissions and Team assignments whenever staffing changes.
- Test schema migrations and bulk imports on a backup before production use.

13. Maintenance and Acceptance

The project includes automated regression tests, chart smoke tests and a PHP/Node parity runner. The current validated baseline reports 213 passing tests, 18 successful chart smoke checks and zero differences in the shared parity fixture. Docker health, login, CRUD access and session persistence across container restarts have also been validated.

```
npm test
npm run smoke:charts
npm run parity
```

14. Support Information

When reporting a problem, provide the timestamp, browser URL, affected module, user role, Docker service status and the last relevant application log lines. Do not include passwords, session cookies, private documents or API credentials in a support request.

Document Status

This runbook describes the Docker deployment and customer workflows validated for the current AP Emlaki 1812-compatible project snapshot.